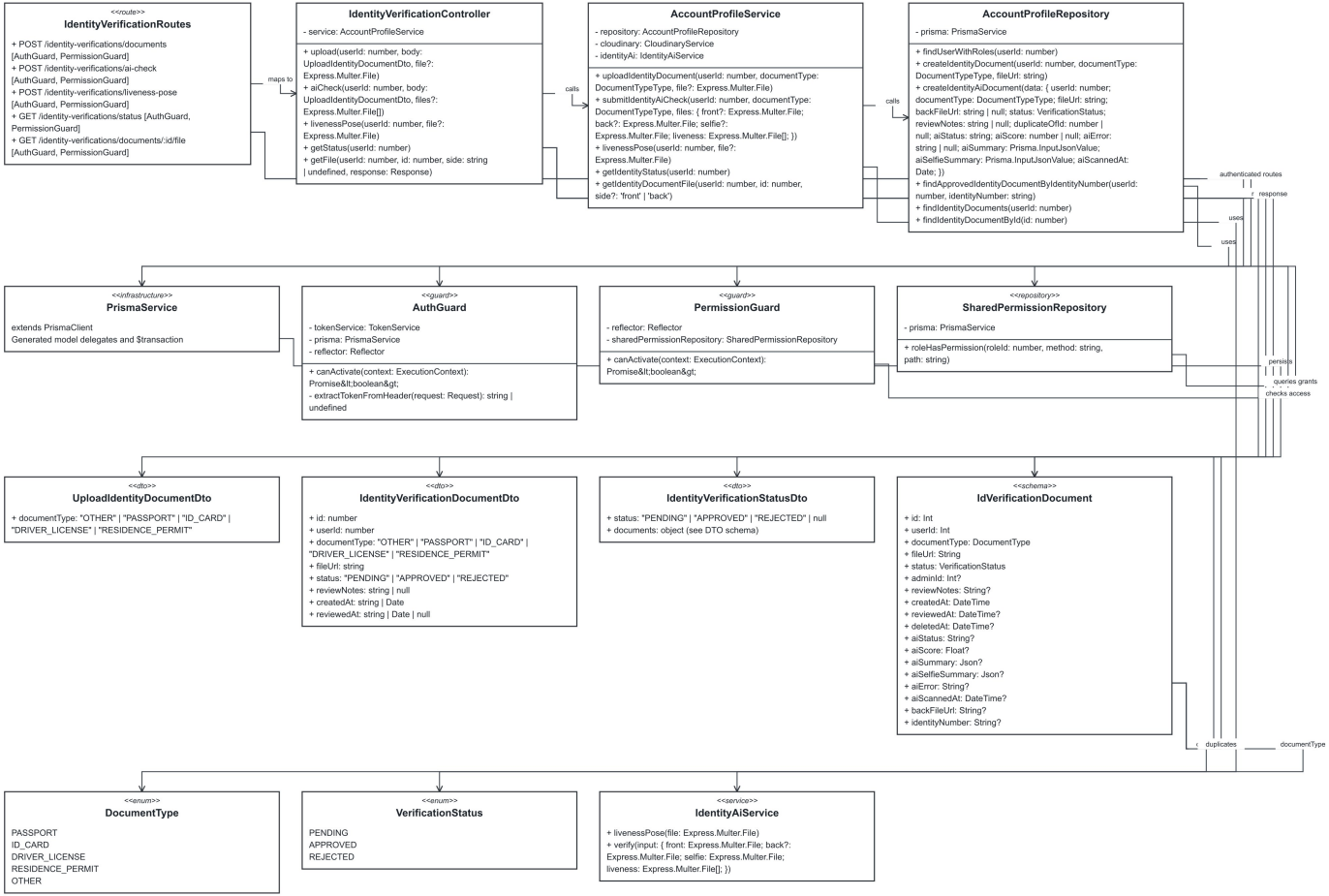

<<route>>
IdentityVerificationRoutes
+ POST /identity-verifications/documents [AuthGuard, PermissionGuard]
+ POST /identity-verifications/ai-check [AuthGuard, PermissionGuard]
+ POST /identity-verifications/liveness-pose [AuthGuard, PermissionGuard]
+ GET /identity-verifications/status [AuthGuard, PermissionGuard]
+ GET /identity-verifications/documents/:id/file [AuthGuard, PermissionGuard]
maps to
IdentityVerificationController
- service: AccountProfileService
+ upload(userId: number, body: UploadIdentityDocumentDto, file?: Express.Multer.File)
+ aiCheck(userId: number, body: UploadIdentityDocumentDto, files?: Express.Multer.File[])
+ livenessPose(userId: number, file?: Express.Multer.File)
+ getStatus(userId: number)
+ getFile(userId: number, id: number, side: string | undefined, response: Response)
calls
AccountProfileService
- repository: AccountProfileRepository
- cloudinary: CloudinaryService
- identityAi: IdentityAiService
+ uploadIdentityDocument(userId: number, documentType: DocumentTypeType, file?: Express.Multer.File)
+ submitIdentityAiCheck(userId: number, documentType: DocumentTypeType, files: { front?: Express.Multer.File; back?: Express.Multer.File; selfie?: Express.Multer.File; liveness: Express.Multer.File[]; })
+ livenessPose(userId: number, file?: Express.Multer.File)
+ getIdentityStatus(userId: number)
+ getIdentityDocumentFile(userId: number, id: number, side?: 'front' | 'back')
calls
AccountProfileRepository
- prisma: PrismaService
+ findUserWithRoles(userId: number)
+ createIdentityDocument(userId: number, documentType: DocumentTypeType, fileUrl: string)
+ createIdentityAiDocument(data: { userId: number; documentType: DocumentTypeType; fileUrl: string; backFileUrl: string | null; status: VerificationStatus; reviewNotes: string | null; duplicateOfId: number | null; aiStatus: string; aiScore: number | null; aiError: string | null; aiSummary: Prisma.InputJsonValue; aiSelfieSummary: Prisma.InputJsonValue; aiScannedAt: Date; })
+ findApprovedIdentityDocumentByIdentityNumber(userId: number, identityNumber: string)
+ findIdentityDocuments(userId: number)
+ findIdentityDocumentById(id: number)
authenticated routes
response
uses
uses
<<infrastructure>>
PrismaService
extends PrismaClient
Generated model delegates and $transaction
<<guard>>
AuthGuard
- tokenService: TokenService
- prisma: PrismaService
- reflector: Reflector
+ canActivate(context: ExecutionContext): Promise<boolean>
- extractTokenFromHeader(request: Request): string | undefined
<<guard>>
PermissionGuard
- reflector: Reflector
- sharedPermissionRepository: SharedPermissionRepository
+ canActivate(context: ExecutionContext): Promise<boolean>
<<repository>>
SharedPermissionRepository
- prisma: PrismaService
+ roleHasPermission(roleId: number, method: string, path: string)
persists
queries grants
checks access
<<dto>>
UploadIdentityDocumentDto
+ documentType: "OTHER" | "PASSPORT" | "ID_CARD" | "DRIVER_LICENSE" | "RESIDENCE_PERMIT"
<<dto>>
IdentityVerificationDocumentDto
+ id: number
+ userId: number
+ documentType: "OTHER" | "PASSPORT" | "ID_CARD" | "DRIVER_LICENSE" | "RESIDENCE_PERMIT"
+ fileUrl: string
+ status: "PENDING" | "APPROVED" | "REJECTED"
+ reviewNotes: string | null
+ createdAt: string | Date
+ reviewedAt: string | Date | null
<<dto>>
IdentityVerificationStatusDto
+ status: "PENDING" | "APPROVED" | "REJECTED" | null
+ documents: object (see DTO schema)
<<schema>>
IdVerificationDocument
+ id: Int
+ userId: Int
+ documentType: DocumentType
+ fileUrl: String
+ status: VerificationStatus
+ adminId: Int?
+ reviewNotes: String?
+ createdAt: DateTime
+ reviewedAt: DateTime?
+ deletedAt: DateTime?
+ aiStatus: String?
+ aiScore: Float?
+ aiSummary: Json?
+ aiSelfieSummary: Json?
+ aiError: String?
+ aiScannedAt: DateTime?
+ backFileUrl: String?
+ identityNumber: String?
duplicates
documentType
<<enum>>
DocumentType
PASSPORT
ID_CARD
DRIVER_LICENSE
RESIDENCE_PERMIT
OTHER
<<enum>>
VerificationStatus
PENDING
APPROVED
REJECTED
<<service>>
IdentityAiService
+ livenessPose(file: Express.Multer.File)
+ verify(input: { front: Express.Multer.File; back?: Express.Multer.File; selfie: Express.Multer.File; liveness: Express.Multer.File[]; })